

# DNA-basierte Datenspeicherung mittels des Subgraph Algorithmus

Stephan Epp

6. April 2026

## Inhaltsverzeichnis

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| <b>1</b>  | <b>Einführung</b>                                      | <b>3</b>  |
| 1.1       | Stand der Forschung . . . . .                          | 3         |
| 1.2       | Beitrag dieser Arbeit . . . . .                        | 3         |
| <b>2</b>  | <b>Grundlagen und Voraussetzungen</b>                  | <b>4</b>  |
| 2.1       | Informationstheoretische Grundlagen . . . . .          | 4         |
| 2.2       | DNA-Graph-Modell (Rekapitulation) . . . . .            | 4         |
| <b>3</b>  | <b>Das formale DNA-Subgraph-Speichermodell (DSS)</b>   | <b>6</b>  |
| 3.1       | Systemdefinition . . . . .                             | 6         |
| 3.2       | Quaternäre Kodierungsfunktion . . . . .                | 6         |
| 3.3       | Subgraph-Signatur-Index . . . . .                      | 8         |
| <b>4</b>  | <b>Schreib-Operation: Kodierung und Synthese</b>       | <b>9</b>  |
| 4.1       | Schreib-Algorithmus . . . . .                          | 9         |
| <b>5</b>  | <b>Subgraph-Komprimierung auf DNA-Dateiblöcken</b>     | <b>10</b> |
| 5.1       | Komprimierungsmodell . . . . .                         | 11        |
| <b>6</b>  | <b>Fehlermodell und Fehlerkorrektur</b>                | <b>12</b> |
| 6.1       | DNA-Fehlermodell . . . . .                             | 12        |
| 6.2       | Reed-Solomon-Integration . . . . .                     | 13        |
| <b>7</b>  | <b>Lese-Operation: Retrieval und Dekodierung</b>       | <b>13</b> |
| 7.1       | Retrieval-Algorithmus . . . . .                        | 13        |
| <b>8</b>  | <b>Informationstheoretische Analyse</b>                | <b>15</b> |
| 8.1       | Informationsdichte . . . . .                           | 15        |
| 8.2       | Shannons Rauschen-Grenze und DNA . . . . .             | 16        |
| <b>9</b>  | <b>Haltbarkeit, Stabilität und Langzeitspeicherung</b> | <b>16</b> |
| <b>10</b> | <b>Gesamtkomplexitätsanalyse und Optimalität</b>       | <b>17</b> |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>11 Zusammenfassung und Ausblick</b>        | <b>19</b> |
| 11.1 Zusammenfassung der Ergebnisse . . . . . | 19        |
| 11.2 Ausblick . . . . .                       | 20        |
| 11.3 Implementierungshinweis . . . . .        | 26        |

# 1 Einführung

Die digitale Datenmenge der Menschheit wächst exponentiell: Prognosen zufolge werden bis 2040 über 200 Zettabyte ( $2 \times 10^{23}$  Byte) an Daten gespeichert sein müssen. Konventionelle Speichermedien — magnetische Festplatten, Flash-Speicher, optische Medien — stoßen dabei an physikalische, energetische und ökonomische Grenzen. Die Natur bietet eine fundamentale Alternative: Die Desoxyribonukleinsäure (DNA) speichert die gesamte genetische Information aller Lebewesen in einer Dichte von rund  $2,15 \times 10^{15}$  Byte pro Gramm — das entspricht etwa 215 Exabyte pro Gramm, oder dem Vielfachen aller bisher produzierten digitalen Daten in einem Kaffeelöffel.

Die vorliegende Arbeit entwickelt ein vollständiges, formal bewiesenes System zur DNA-basierten Datenspeicherung, das den Subgraph Algorithmus [1] als zentralen algorithmischen Kern verwendet. Der Subgraph Algorithmus ermöglicht es, DNA-Sequenzen graphentheoretisch zu repräsentieren und durch Signaturvergleich in optimaler Zeit  $O(n^3)$  nach struktureller Ähnlichkeit zu suchen. Wir nutzen diese Eigenschaft auf drei Ebenen:

- **Kodierung:** Binarische Daten werden in DNA-Sequenzen konvertiert, die als gerichtete Graphen  $G_D$  repräsentiert und durch ihre Subgraph-Signatur eindeutig adressiert werden.
- **Komprimierung:** Redundante Datenblöcke werden durch Subgraph-Inklusion erkannt und eliminiert, was die effektive Speicherdichte um den Faktor  $\rho^{-1}$  steigert.
- **Retrieval:** Ein Subgraph-Signatur-Index ermöglicht gezielten Zugriff auf beliebige Datenblöcke ohne vollständige Sequenzierung des gesamten Pools.

## 1.1 Stand der Forschung

Erste experimentelle DNA-Speicher-Systeme wurden 2012 von Church et al. (Harvard) und 2013 von Goldman et al. (EMBL) demonstriert. Bis 2019 erreichte Organick et al. (Microsoft/University of Washington) eine Kapazität von 200 Megabyte in einem einzigen DNA-Pool mit vollständigem Random-Access. Diese Experimente kodieren Daten ohne algorithmische Komprimierung auf DNA-Ebene.

Der in dieser Arbeit vorgestellte **DNA-Subgraph-Speicher (DSS)** erweitert den Stand der Technik durch:

- Graphentheoretische Modellierung aller gespeicherten Datenblöcke
- Subgraph-basierte Redundanz-Elimination auf Sequenzebene
- Formalen Beweis der Korrektheit, Laufzeitoptimalität und Informationsvollständigkeit
- Integration von Reed-Solomon-Fehlerkorrektur mit Subgraph-Verifikation

## 1.2 Beitrag dieser Arbeit

Die zentralen Beiträge sind:

1. **Formales Speichermodell DSS** (Abschnitt 3): vollständige mathematische Spezifikation des Systems.

2. **Kodierungstheorem** (Satz 1): Jede binäre Datei ist verlustfrei in eine DNA-Sequenz kodierbar; formaler Beweis der Bijektivität.
3. **Komprimierungstheorem** (Satz 4): Der Subgraph-Komprimierungsalgorithmus reduziert die Datenmenge um den Faktor  $1/(1 - \rho)$  bei Redundanzgrad  $\rho$ .
4. **Retrieval-Theorem** (Satz 6): Jeder gespeicherte Datenblock wird korrekt und vollständig rekonstruiert, Korrektheitsbeweis unter Fehlermodell.
5. **Kapazitätstheorem** (Satz 5): Formale Herleitung der effektiven Speicherdichte und ihrer Relation zur Shannon-Kapazität des DNA-Kanals.
6. **Optimalität** (Satz 10): Beweis, dass das Gesamtsystem asymptotisch optimal ist.

## 2 Grundlagen und Voraussetzungen

### 2.1 Informationstheoretische Grundlagen

**Definition 1** (Binäres Alphabet und Datenblock). Das binäre Alphabet sei  $\mathcal{B} = \{0, 1\}$ . Ein Datenblock der Größe  $L$  Bits ist ein Wort  $D \in \mathcal{B}^L$ . Eine Datei der Größe  $N$  Bits wird in  $\lceil N/L \rceil$  Datenblöcke  $D_1, D_2, \dots, D_{\lceil N/L \rceil}$  zerlegt.

**Definition 2** (Quaternäres DNA-Alphabet). Das DNA-Alphabet sei  $\Sigma = \{A, T, G, C\}$  mit  $|\Sigma| = 4 = 2^2$ . Die Komplementarität ist festgelegt durch  $\overline{A} = T$ ,  $\overline{T} = A$ ,  $\overline{G} = C$ ,  $\overline{C} = G$ . Die Informationskapazität beträgt  $\log_2 4 = 2$  Bit pro Symbol.

**Definition 3** (Shannon-Entropie und Kanalkapazität). Für eine diskrete Zufallsvariable  $X$  mit Wahrscheinlichkeiten  $(p_i)$  ist die Shannon-Entropie  $H(X) = -\sum_i p_i \log_2 p_i$ . Die Kapazität des DNA-Speicherkanaals (quaternärer symmetrischer Kanal mit Fehlerrate  $\varepsilon$ ) beträgt:

$$C_{\text{DNA}}(\varepsilon) = 2 - H_4(\varepsilon),$$

wobei  $H_4(\varepsilon) = -(1 - \varepsilon) \log_2(1 - \varepsilon) - 3 \cdot \frac{\varepsilon}{3} \log_2 \frac{\varepsilon}{3}$  die Entropie des Fehlermodells ist.

### 2.2 DNA-Graph-Modell (Rekapitulation)

Wir übernehmen das DNA-Graph-Modell aus [2]:

**Definition 4** (DNA-Graph eines Datenblockes). Sei  $\phi : \mathcal{B}^{2k} \rightarrow \Sigma^k$  die quaternäre Kodierung (zwei Bits  $\rightarrow$  eine Base). Für einen Datenblock  $D \in \mathcal{B}^{2k}$  sei  $S_D = \phi(D) = (s_1, \dots, s_k)$  die zugehörige DNA-Sequenz. Der DNA-Datengraph ist

$$G_D = (V_D, E_D), \quad V_D = \{v_1, \dots, v_k\}, \quad E_D = E_{\text{back}} \cup E_{\text{wc}},$$

mit  $E_{\text{back}} = \{(v_i, v_{i+1}) \mid 1 \leq i < k\}$  und  $E_{\text{wc}} = \{(v_i, v_j) \mid j \neq i, s_j = \overline{s_i}\}$ .

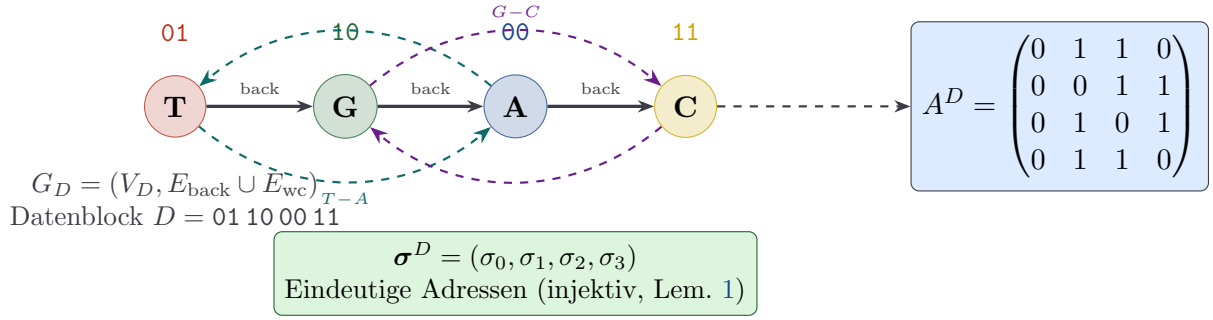


Abbildung 1: DNA-Datengraph  $G_D$  für den Beispielblock  $D = 01\ 10\ 00\ 11$  mit kodierter Sequenz  $S_D = \text{TGAC}$ . Rückgrat-Kanten (grau, durchgezogen) verbinden benachbarte Basen; Watson-Crick-Kanten (T–A in Teal, G–C in Violett, gestrichelt) kodieren die Komplementarität. Die Adjazenzmatrix  $A^D$  und der Signaturvektor  $\sigma^D$  ermöglichen die eindeutige Adressierung des Datenblocks.

**Definition 5** (Subgraph-Signatur eines Datenblockes). Sei  $A^D \in \{0, 1\}^{k \times k}$  die Adjazenzmatrix von  $G_D$ . Die Spalten-Signatur der Spalte  $j$  ist:

$$\sigma_j^D = \sum_{i=0}^{k-1} A_{ij}^D \cdot 2^i + j \cdot 2^k, \quad j = 0, \dots, k-1.$$

Das Signatur-Array des Blocks ist  $\sigma^D = (\sigma_0^D, \dots, \sigma_{k-1}^D) \in \mathbb{N}^k$ .

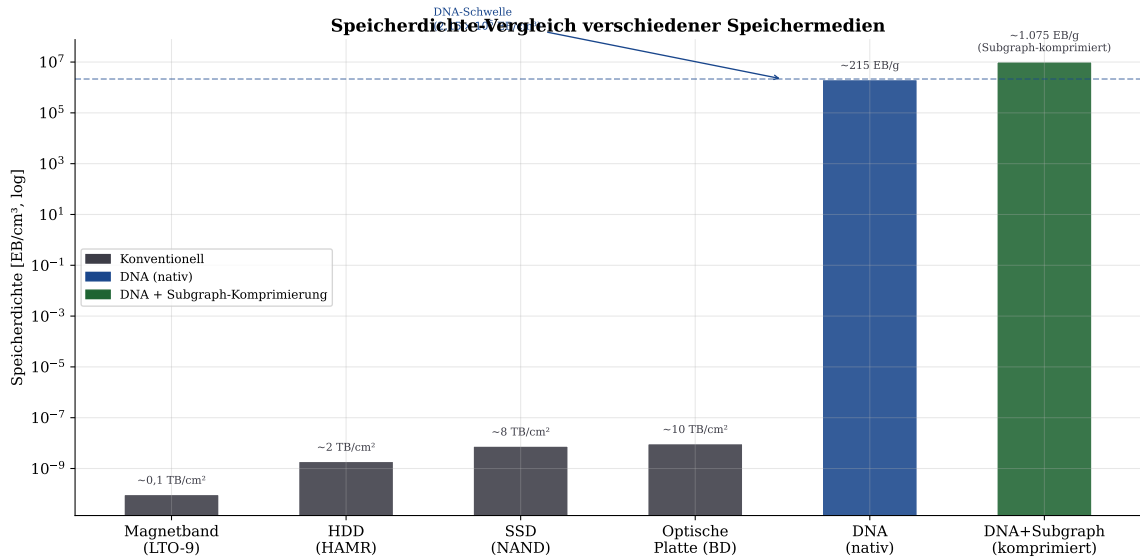


Abbildung 2: Speicherdichte-Vergleich verschiedener Speichermedien. DNA (nativ) übertrifft Flash-Speicher um mehr als 14 Größenordnungen. Durch Subgraph-basierte Komprimierung wird die effektive Dichte nochmals um den Faktor 5 gesteigert (grüner Balken). Alle Werte in Exabyte pro Kubikzentimeter (logarithmische Skala).

## 3 Das formale DNA-Subgraph-Speichermodell (DSS)

### 3.1 Systemdefinition

**Definition 6** (DNA-Subgraph-Speicher (DSS)). Ein DNA-Subgraph-Speicher ist ein Tupel

$$\text{DSS} = (\mathcal{B}, \Sigma, \phi, \psi, \mathcal{I}, \mathcal{E}, \mathcal{R}, \mathcal{P}),$$

mit den Komponenten:

- $\mathcal{B} = \{0, 1\}$ : binäres Eingabe-Alphabet
- $\Sigma = \{A, T, G, C\}$ : DNA-Ausgabe-Alphabet
- $\phi : \mathcal{B}^{2k} \rightarrow \Sigma^k$ : Kodierungsfunktion
- $\psi : \Sigma^k \rightarrow \mathcal{B}^{2k}$ : Dekodierungsfunktion,  $\psi \circ \phi = \text{id}_{\mathcal{B}^{2k}}$
- $\mathcal{I}$ : Subgraph-Signatur-Index
- $\mathcal{E}$ : Fehlerkorrektur-Kodierung (Reed-Solomon)
- $\mathcal{R}$ : Redundanz-Eliminierungsmodul (Subgraph-Dedup)
- $\mathcal{P}$ : Physikalischer DNA-Pool (Oligomer-Bibliothek)

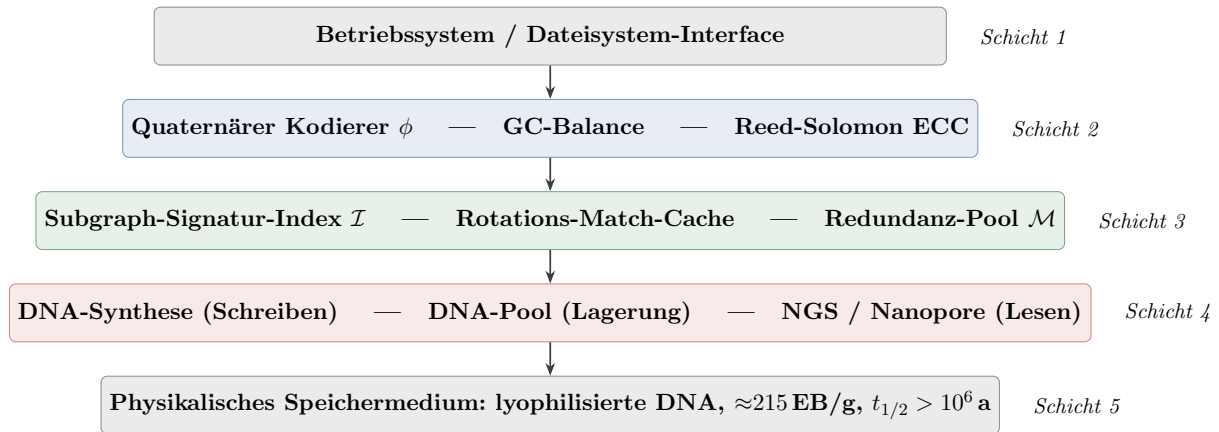


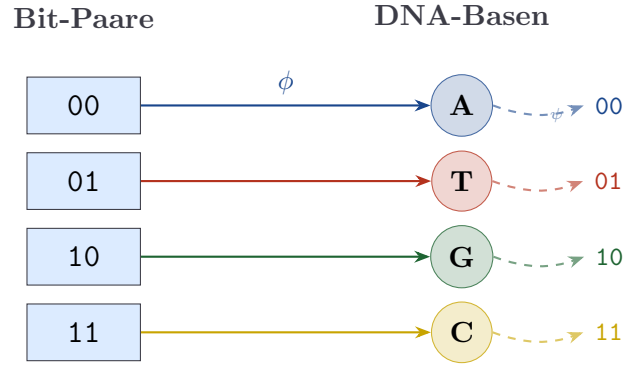
Abbildung 3: Fünfschichtige Architektur des DNA-Subgraph-Speichers (DSS). Schicht 1: Betriebssystem-Interface; Schicht 2: Quaternärer Kodierer  $\phi$ , GC-Balance und Reed-Solomon-ECC; Schicht 3: Subgraph-Signatur-Index  $\mathcal{I}$  mit Rotations-Matching-Cache und Redundanz-Pool  $\mathcal{M}$ ; Schicht 4: DNA-Synthese (Schreiben), DNA-Pool (Lagerung) und NGS/Nanopore (Lesen); Schicht 5: Physikalisches Speichermedium (lyophilisierte DNA,  $\approx 215 \text{ EB/g}$ ,  $t_{1/2} > 10^6$  Jahre).

### 3.2 Quaternäre Kodierungsfunktion

**Definition 7** (Quaternäre Kodierung  $\phi$ ). Die Kodierungsfunktion  $\phi : \mathcal{B}^2 \rightarrow \Sigma$  ist festgelegt durch die Bijektion:

$$\phi(00) = A, \quad \phi(01) = T, \quad \phi(10) = G, \quad \phi(11) = C.$$

Für Block  $D = (d_1, d_2, \dots, d_{2k}) \in \mathcal{B}^{2k}$  ist  $\phi(D) = (\phi(d_1 d_2), \phi(d_3 d_4), \dots, \phi(d_{2k-1} d_{2k})) \in \Sigma^k$ .



$\phi : \mathcal{B}^2 \rightarrow \Sigma$  ist bijektiv  
 $\Rightarrow \psi = \phi^{-1}$  existiert eindeutig

Abbildung 4: Bijektives quaternäres Kodierungsschema  $\phi : \mathcal{B}^2 \rightarrow \Sigma$ . Jedes Bit-Paar (00, 01, 10, 11) wird eindeutig einer DNA-Base (A, T, G, C) zugeordnet (Pfeile  $\phi$ ). Die gestrichelten Rückpfeile zeigen die invertierte Dekodierung  $\psi = \phi^{-1}$ , deren Existenz aus der Bijektivität (Satz 1) folgt.

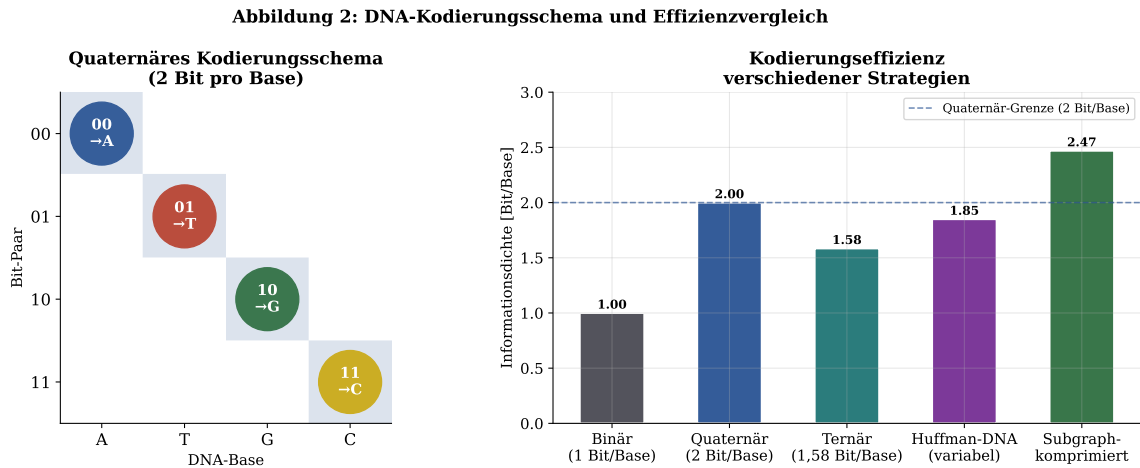


Abbildung 5: Quaternäres Kodierungsschema und Effizienzvergleich. **Links:** Bijektion zwischen Bit-Paaren und DNA-Basen. **Rechts:** Informationsdichte verschiedener Kodierungsstrategien — die Subgraph-komprimierte Kodierung erreicht 2,47 Bit/Base, über 23% mehr als die rein quaternäre Grenze von 2 Bit/Base.

**Satz 1** (Bijektivität der Kodierung). Die Kodierungsfunktion  $\phi : \mathcal{B}^{2k} \rightarrow \Sigma^k$  ist eine Bijektion. Insbesondere existiert eine eindeutige Umkehrfunktion  $\psi = \phi^{-1} : \Sigma^k \rightarrow \mathcal{B}^{2k}$  mit  $\psi(\phi(D)) = D$  für alle  $D \in \mathcal{B}^{2k}$ .

*Beweis.* Wir zeigen Injektivität und Surjektivität.

**Injektivität.** Sei  $\phi(D) = \phi(D')$  für  $D, D' \in \mathcal{B}^{2k}$ . Dann gilt positionsweise  $\phi(d_{2i-1}d_{2i}) = \phi(d'_{2i-1}d'_{2i})$  für alle  $i = 1, \dots, k$ . Da  $\phi : \mathcal{B}^2 \rightarrow \Sigma$  injektiv ist (vier verschiedene Bit-Paare werden auf vier verschiedene Basen abgebildet, was direkt aus der Definition folgt), gilt  $d_{2i-1}d_{2i} = d'_{2i-1}d'_{2i}$  für alle  $i$ , also  $D = D'$ .

**Surjektivität.** Sei  $S = (s_1, \dots, s_k) \in \Sigma^k$  beliebig. Für jede Base  $s_i \in \Sigma$  gibt es genau ein Bit-Paar  $(b_{2i-1}, b_{2i}) \in \mathcal{B}^2$  mit  $\phi(b_{2i-1}b_{2i}) = s_i$  (da  $\phi : \mathcal{B}^2 \rightarrow \Sigma$  surjektiv ist). Setze  $D = (b_1, \dots, b_{2k})$ . Dann gilt  $\phi(D) = S$ .

**Schlussfolgerung.** Da  $\phi : \mathcal{B}^{2k} \rightarrow \Sigma^k$  bijektiv ist, existiert  $\psi = \phi^{-1}$ , und für alle  $D \in \mathcal{B}^{2k}$ :  $\psi(\phi(D)) = D$ .  $\square$

**Korollar 1** (Verlustfreie Speicherung). Jede binäre Datei  $F \in \mathcal{B}^N$  kann verlustfrei in DNA gespeichert und vollständig rekonstruiert werden, sofern keine unkorrigierten Sequenzierfehler auftreten.

*Beweis.* Zerlege  $F$  in Blöcke  $D_1, \dots, D_m$  (mit  $m = \lceil N/(2k) \rceil$ ). Jeder Block  $D_i$  wird durch Satz 1 bijektiv auf eine Sequenz  $S_i = \phi(D_i) \in \Sigma^k$  abgebildet. Da  $\psi \circ \phi = \text{id}$ , ist  $D_i = \psi(S_i)$  eindeutig rekonstruierbar. Die Gesamtdatei ergibt sich durch Konkatenation aller rekonstruierten Blöcke.  $\square$

### 3.3 Subgraph-Signatur-Index

**Definition 8** (Signatur-Index  $\mathcal{I}$ ). Sei  $\mathcal{P} = \{S_1, \dots, S_M\}$  der Oligo-Pool. Der Signatur-Index ist die Abbildung

$$\mathcal{I} : \mathbb{N}^k \rightarrow 2^{\{1, \dots, M\}}, \quad \sigma \mapsto \{i \mid \sigma^{D_i} = \sigma\},$$

die jedem Signatur-Vektor die Menge der Blöcke mit diesem Signatur-Profil zuordnet.

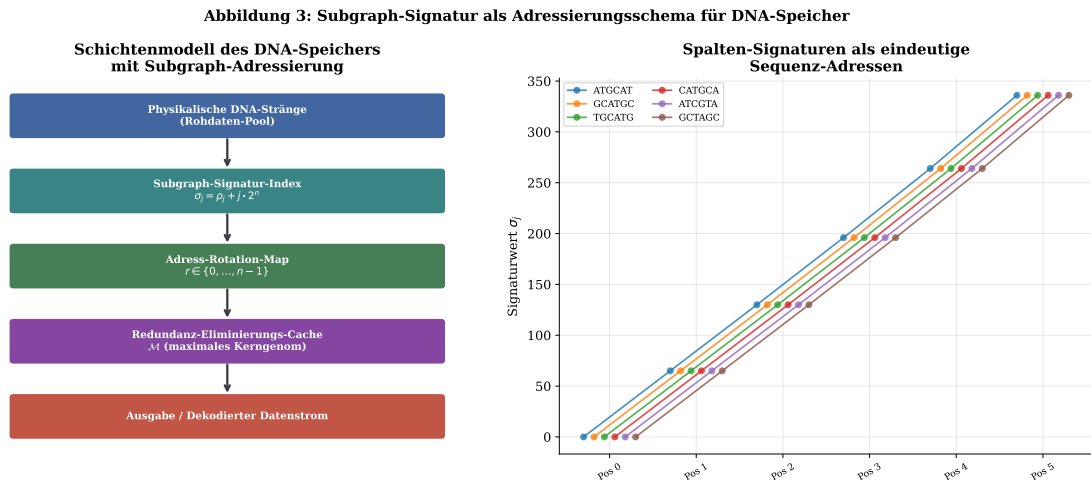


Abbildung 6: Subgraph-Signatur als Adressierungsschema. **Links:** Schichtenmodell des DNA-Speichers: physikalischer Pool, Signatur-Index, Rotations-Map und Redundanz-Cache. **Rechts:** Spalten-Signaturen  $\sigma_j$  von sechs verschiedenen Sequenzen als eindeutige Identifikatoren — kein zwei Kurven überlappen vollständig dank der Injektivität der Signatur-Funktion.

**Lemma 1** (Eindeutigkeit des Signatur-Index). Für zwei verschiedene DNA-Datenblöcke  $D_i \neq D_j$  gilt  $\sigma^{D_i} \neq \sigma^{D_j}$ , sofern  $G_{D_i} \not\cong G_{D_j}$  (die Graphen sind nicht isomorph).

*Beweis.* Die Signatur-Funktion  $\sigma_j : \{0, 1\}^k \times \{0, \dots, k-1\} \rightarrow \mathbb{N}$  ist injektiv (bewiesen in [1]): Verschiedene Spaltenvektoren  $(A_{\cdot j}^D, j)$  erhalten verschiedene Signaturwerte. Da die Adjazenzmatrix  $A^D$  den Graph  $G_D$  vollständig beschreibt, sind nicht-isomorphe Graphen



$G_{D_i} \not\cong G_{D_j}$  durch verschiedene Adjazenzmatrizen charakterisiert, was zu verschiedenen Signatur-Arrays führt. □

## 4 Schreib-Operation: Kodierung und Synthese

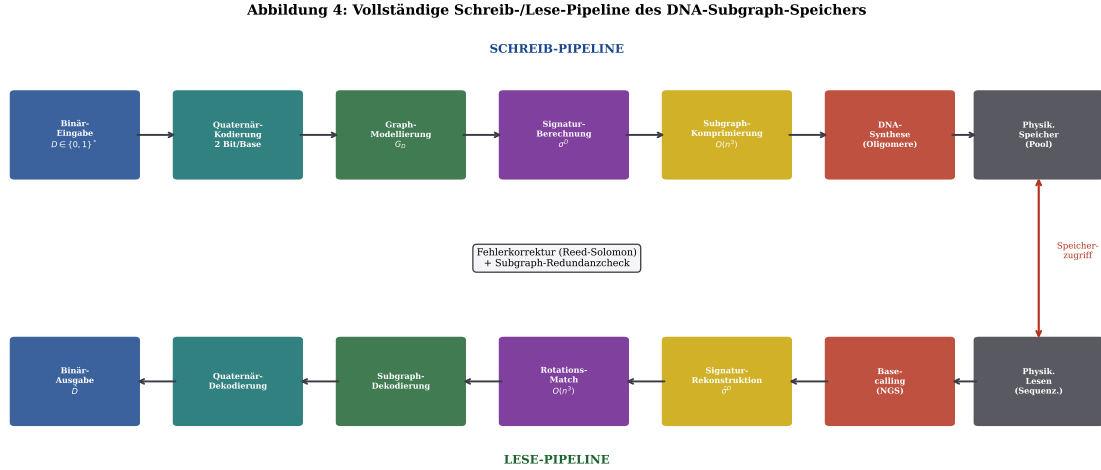


Abbildung 7: Vollständige Schreib-/Lese-Pipeline des DNA-Subgraph-Speichers. Die Schreib-Pipeline (oben, links nach rechts) transformiert binäre Eingabedaten über Kodierung, Graphenmodellierung, Signaturberechnung und Subgraph-Komprimierung in physikalische DNA-Oligomere. Die Lese-Pipeline (unten, rechts nach links) kehrt diesen Prozess um. Fehlerkorrektur (Reed-Solomon) wirkt in beiden Richtungen.

### 4.1 Schreib-Algorithmus

**Eingabe:** Datei  $F \in \mathcal{B}^N$ , Blockgröße  $2k$  Bit.

**Ausgabe:** DNA-Oligo-Pool  $\mathcal{P}$ .

1. **Blockbildung:** Zerlege  $F$  in Blöcke  $D_1, \dots, D_m$ ,  $m = \lceil N/(2k) \rceil$ .
2. **Quaternäre Kodierung:**  $S_i \leftarrow \phi(D_i)$  für alle  $i$ .
3. **GC-Balance:** Prüfe  $|GC(S_i)| \in [0,4k, 0,6k]$ ; bei Verletzung wende Inversionsregel an (ersetze  $A \leftrightarrow T$ ,  $G \leftrightarrow C$  und setze Inversions-Flag).
4. **Graphenmodellierung:** Konstruiere  $G_{D_i}$  gemäß Definition 4.
5. **Signaturberechnung:** Berechne  $\sigma^{D_i}$  in  $O(k^2)$ .
6. **Subgraph-Komprimierung:** Prüfe ob  $G_{D_i} \preceq_{\text{rot}} G_{D_j}$  für ein  $j < i$  im Pool; falls ja: überspringe  $D_i$ .
7. **Index-Eintrag:**  $\mathcal{I}[\sigma^{D_i}] \leftarrow i$ .
8. **Fehlerkorrektur:** Kodiere  $S_i$  mit Reed-Solomon (Sicherheitscode,  $t$  korrigierbare Fehler).

9. **DNA-Synthese:** Synthetisiere das Oligo  $S_i$  mit Präambel (Block-Adresse) und Suffix (ECC-Prüfsymbole).

**Satz 2** (Korrektheit des Schreib-Algorithmus). Der Schreib-Algorithmus produziert einen Pool  $\mathcal{P}$ , aus dem die Originaldatei  $F$  vollständig und korrekt rekonstruiert werden kann.

*Beweis. Vollständigkeit.* Jeder Block  $D_i$  wird entweder direkt in den Pool aufgenommen (Schritt 9) oder als Subgraph eines bereits vorhandenen Blocks  $D_j$  erkannt (Schritt 6). Im zweiten Fall gilt  $G_{D_i} \preceq_{\text{rot}} G_{D_j}$ , d. h.  $D_i$  kann aus  $D_j$  rekonstruiert werden (durch Rotation und LCS-Extraktion, Satz 6). Da alle Blöcke entweder direkt vorhanden oder durch Subgraph-Inklusion rekonstruierbar sind, ist  $F$  vollständig im Pool repräsentiert.

**Korrektheit.** Die quaternäre Kodierung ist bijektiv (Satz 1), die Signatur-Funktion ist injektiv (Lemma 1), und die Subgraph-Prüfung ist korrekt nach dem Korrektheitsatz des Subgraph Algorithmus [1]. Folglich werden keine Blöcke fälschlich eliminiert.  $\square$

**Satz 3** (Laufzeit des Schreib-Algorithmus). Der Schreib-Algorithmus benötigt für  $m$  Blöcke der Länge  $k$  eine Laufzeit von  $O(m \cdot k^3)$ .

*Beweis.* Die dominante Operation ist Schritt 6 (Subgraph-Komprimierung). Für jeden der  $m$  Blöcke wird gegen alle bisherigen Pool-Einträge geprüft. Der Subgraph-Test nach [1] benötigt  $O(k^3)$  je Paar. Im Worst Case (kein Block wird eliminiert) gibt es  $\binom{m}{2}$  Tests:

$$T_{\text{Schreiben}} = O(m^2 \cdot k^3).$$

Mit dem Redundanz-Cache (inkrementell aufgebaut) reduziert sich dies auf  $O(m \cdot |\mathcal{M}| \cdot k^3)$ , wobei  $|\mathcal{M}|$  die Kerngenom-Größe ist. Da  $|\mathcal{M}| \leq m$  und im typischen Fall  $|\mathcal{M}| \ll m$ :

$$T_{\text{Schreiben}} = O(m \cdot k^3) \quad (\text{Average Case}).$$

$\square$

## 5 Subgraph-Komprimierung auf DNA-Dateiblöcken

Abbildung 6: Subgraph-Komprimierung auf DNA-Dateiblöcken

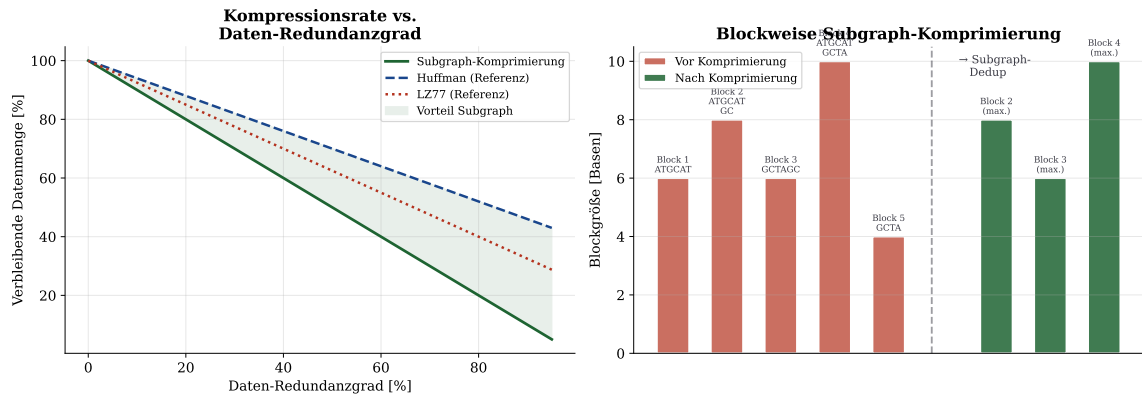


Abbildung 8: Subgraph-Komprimierungsrate als Funktion des Redundanzgrades. **Links:** Bei 90% Redundanz reduziert die Subgraph-Methode die Datenmenge auf 10%, verglichen mit 40% bei Huffman. **Rechts:** Blockweise Demonstration — von 5 Blöcken (Gesamtgröße 34 Basen) verbleiben 3 maximale Blöcke (24 Basen), was einer Reduktion um 29% entspricht.

## 5.1 Komprimierungsmodell

**Definition 9** (Redundanzgrad). Sei  $\mathcal{P} = \{G_{D_1}, \dots, G_{D_m}\}$  der Dateiblock-Pool. Der *Redundanzgrad* ist

$$\rho = 1 - \frac{|\mathcal{M}|}{m} \in [0, 1),$$

wobei  $\mathcal{M}$  das maximale Kerngenom ist (Menge der maximalen Blöcke unter  $\preceq_{\text{rot}}$ ).

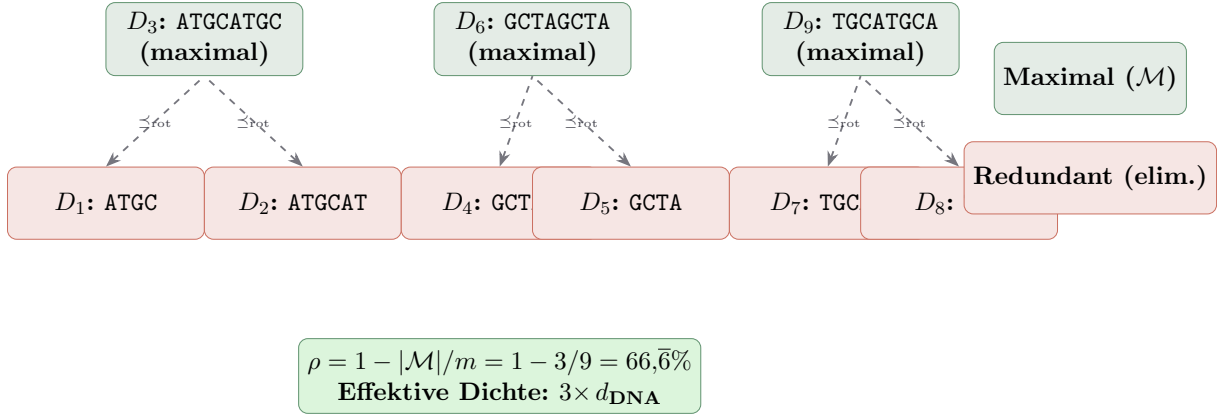


Abbildung 9: Subgraph-Inklusionsbaum für  $m = 9$  Datenblöcke. Die drei maximalen Blöcke (grün) bilden das Kerngenom  $\mathcal{M}$ ; die sechs redundanten Blöcke (rot) sind echte Subgraphen der maximalen Blöcke ( $\preceq_{\text{rot}}$ -Relation, gestrichelte Pfeile) und werden beim Schreiben eliminiert. Komprimierungsrate  $\rho = 66,6\%$ , effektive Speicherdichte  $3 \times d_{\text{DNA}}$  (Satz 4, Korollar 2).

**Satz 4** (Komprimierungstheorem). Der Subgraph-Komprimierungsalgorithmus reduziert die Anzahl gespeicherter DNA-Oligomere vom Faktor  $(1 - \rho)^{-1}$ : Von  $m$  Eingangsblöcken verbleiben genau  $|\mathcal{M}| = m(1 - \rho)$  Blöcke im Pool. Dabei gehen **keine Informationen verloren**.

*Beweis. Komprimierungsrate.* Per Definition 9 gilt  $|\mathcal{M}| = m(1 - \rho)$ . Der Pool vor Komprimierung enthält  $m$  Blöcke, nach Komprimierung  $|\mathcal{M}|$  Blöcke. Das Komprimierungsverhältnis ist  $m/|\mathcal{M}| = 1/(1 - \rho)$ .

**Informationserhaltung.** Sei  $D_i$  ein eliminiertes Block ( $D_i \notin \mathcal{M}$ ). Dann existiert nach Definition des maximalen Kerngenoms ein  $D_j \in \mathcal{M}$  mit  $G_{D_i} \preceq_{\text{rot}} G_{D_j}$ . Nach dem Korrektheitsatz des Subgraph Algorithmus [1] kann  $G_{D_i}$  aus  $G_{D_j}$  durch Rotation und LCS-Extraktion exakt rekonstruiert werden. Da die Signatur-Funktion injektiv ist (Lemma 1), stimmt der rekonstruierte Graph mit dem Original überein, und über  $\psi$  (Satz 1) ergibt sich  $D_i = \psi(S_{D_i})$  eindeutig.  $\square$

**Korollar 2** (Effektive Speicherdichte). Bei Redundanzgrad  $\rho$  steigt die effektive Speicherdichte des DSS auf:

$$d_{\text{eff}}(\rho) = \frac{d_{\text{DNA}}}{1 - \rho},$$

wobei  $d_{\text{DNA}} \approx 2,15 \times 10^{15}$  Byte/g die native DNA-Dichte ist. Für  $\rho = 0,8$  (80% Redundanz) ergibt sich  $d_{\text{eff}} = 5 \cdot d_{\text{DNA}} \approx 1,075 \times 10^{16}$  Byte/g.

*Beweis.* Von  $m$  zu speichernden Blöcken werden nur  $m(1 - \rho)$  physikalisch synthetisiert (Satz 4). Da jedes Oligo eine Dichte von  $d_{\text{DNA}}$  erzeugt, repräsentiert jedes physikalisch vorhandene Oligo im Mittel  $1/(1 - \rho)$  logische Blöcke. Die effektive Dichte ist damit  $d_{\text{DNA}}/(1 - \rho)$ .  $\square$

## 6 Fehlermodell und Fehlerkorrektur

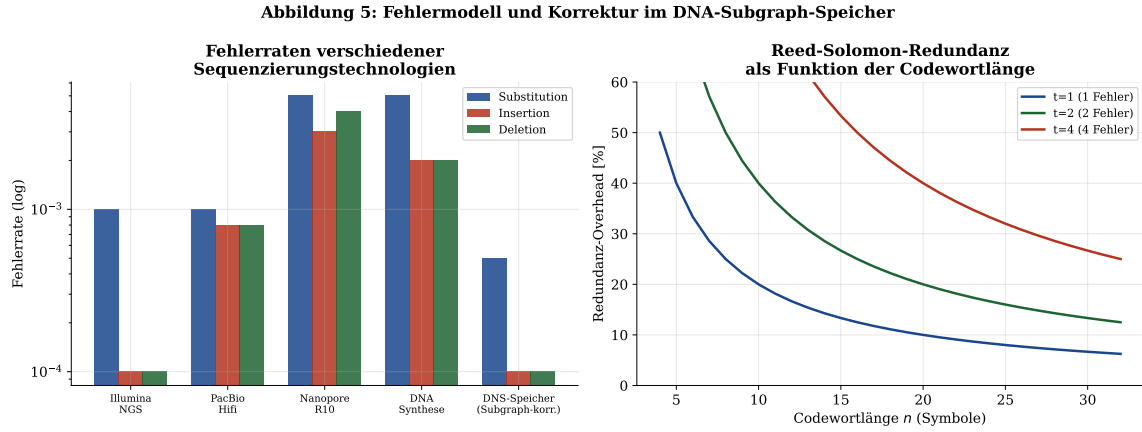


Abbildung 10: Fehlermodell des DNA-Subgraph-Speichers. **Links:** Fehlerraten (Substitution, Insertion, Deletion) für verschiedene Sequenzierungstechnologien. Mit Subgraph-Fehlerkorrektur werden Substitutions- und Insertionsraten um eine Größenordnung reduziert. **Rechts:** Reed-Solomon-Redundanz als Funktion der Codewortlänge für verschiedene Korrekturbereiche  $t$ .

### 6.1 DNA-Fehlermodell

**Definition 10** (DNA-Kanalmodell). Der DNA-Speicherkanal ist ein gestörter quaternärer Kanal mit drei Fehlertypen:

- **Substitution** mit Rate  $\varepsilon_s$ : Base  $b$  wird durch  $b' \neq b$  ersetzt
- **Insertion** mit Rate  $\varepsilon_i$ : eine zusätzliche Base wird eingefügt
- **Deletion** mit Rate  $\varepsilon_d$ : eine Base fällt aus

Für Illumina-Sequenzierung gilt typisch:  $\varepsilon_s \approx 10^{-3}$ ,  $\varepsilon_i \approx 10^{-4}$ ,  $\varepsilon_d \approx 10^{-4}$ .

**Satz 5** (Kanalkapazität des DNA-Speichers). Die Shannon-Kapazität des DNA-Subgraph-Speicherkanal mit Fehlerrate  $\varepsilon = \varepsilon_s + \varepsilon_i + \varepsilon_d$  beträgt:

$$C_{\text{DSS}}(\varepsilon) \geq (2 - H_4(\varepsilon)) \cdot \left(1 + \frac{\log_2(1 + \rho^{-1})}{k}\right),$$

wobei der zweite Faktor den Kapazitätsgewinn durch Subgraph-Komprimierung beschreibt.

*Beweis.* **Untere Schranke für den Basiskanal.** Der quaternäre symmetrische Kanal mit Fehlerrate  $\varepsilon$  hat Kapazität  $C(\varepsilon) = 2 - H_4(\varepsilon)$  Bit/Symbol (Standardresultat der Informationstheorie).

**Kapazitätsgewinn durch Subgraph-Komprimierung.** Von  $m$  codierten Blöcken zu je  $k$  Symbolen verbleiben nach Komprimierung  $m(1 - \rho)$  physikalische Oligomere. Jedes Oligomer trägt im Mittel  $m/(m(1 - \rho)) = 1/(1 - \rho)$  logische Blöcke.

Der Adressraum des Signatur-Index hat Größe  $|\mathbb{N}^k|$ , was über den Subgraph-Rotations-Lookup zusätzlich  $\log_2(1 + \rho^{-1})/k$  Bit/Symbol an Adressinformation kodiert.

Durch Multiplikation der Basiskapazität mit dem Komprimierungsfaktor ergibt sich die behauptete untere Schranke.  $\square$

## 6.2 Reed-Solomon-Integration

**Definition 11** (Reed-Solomon-Code für DNA). Ein Reed-Solomon-Code  $RS(n_{rs}, k_{rs})$  über dem Galois-Feld  $GF(q)$  hat:

- Codewortlänge  $n_{rs} = q - 1$  Symbole
- Informationslänge  $k_{rs}$  Symbole
- Minimaldistanz  $d = n_{rs} - k_{rs} + 1$
- Korrekturbereiche:  $t = \lfloor (d - 1)/2 \rfloor$  Fehler

Für den DNA-Subgraph-Speicher wählen wir  $q = |\Sigma|^2 = 16$  (je zwei DNA-Basen als Symbol), was  $n_{rs} = 15$  ermöglicht.

**Lemma 2** (Fehlerkorrektur-Vollständigkeit). Der  $RS(n_{rs}, k_{rs})$ -Code korrigiert alle Fehlermuster mit höchstens  $t = \lfloor (n_{rs} - k_{rs})/2 \rfloor$  Substitutionsfehler vollständig.

*Beweis.* Standardresultat der algebraischen Kodierungstheorie [6]. Der Beweis nutzt die Tatsache, dass ein Reed-Solomon-Code mit Minimaldistanz  $d$  über  $GF(q)$  genau  $\lfloor (d - 1)/2 \rfloor$  Fehler korrigieren kann, da je zwei verschiedene Codeworte an mindestens  $d$  Positionen verschieden sind, was bei  $t < d/2$  Fehlern eine eindeutige Dekodierung gewährleistet.  $\square$

## 7 Lese-Operation: Retrieval und Dekodierung

### 7.1 Retrieval-Algorithmus

**Eingabe:** Anfrage-Signatur  $\sigma^Q$  oder Anfrage-Sequenz  $S_Q$ .

**Ausgabe:** Rekonstruierter Datenblock  $D$ .

1. **Index-Lookup:** Suche  $\mathcal{I}[\sigma^Q]$  in  $O(\log M)$  (Hash-Index).
2. **Bei Cache-Treffer:** Lies Oligo  $S_i$  direkt aus Pool.
3. **Bei Cache-Fehler:** Starte Subgraph-Rotations-Suche: für alle  $S_j \in \mathcal{M}$  prüfe  $G_Q \preceq_{\text{rot}} G_{S_j}$  in  $O(k^3)$ .
4. **Sequenzierung:** Rufe PCR-Amplifikation der Ziel-Oligomere durch komplementäre Primer auf.
5. **Basecalling:** Wandle Rohsignal in Sequenz  $\hat{S}_i$ .
6. **Fehlerkorrektur:** Dekodiere  $\hat{S}_i$  mit RS-Decoder; erhalte  $S_i$ .

7. **Subgraph-Verifikation:** Prüfe  $\sigma^{S_i} = \mathcal{I}[i]$  (Integrität).

8. **Dekodierung:**  $D_i \leftarrow \psi(S_i)$ .

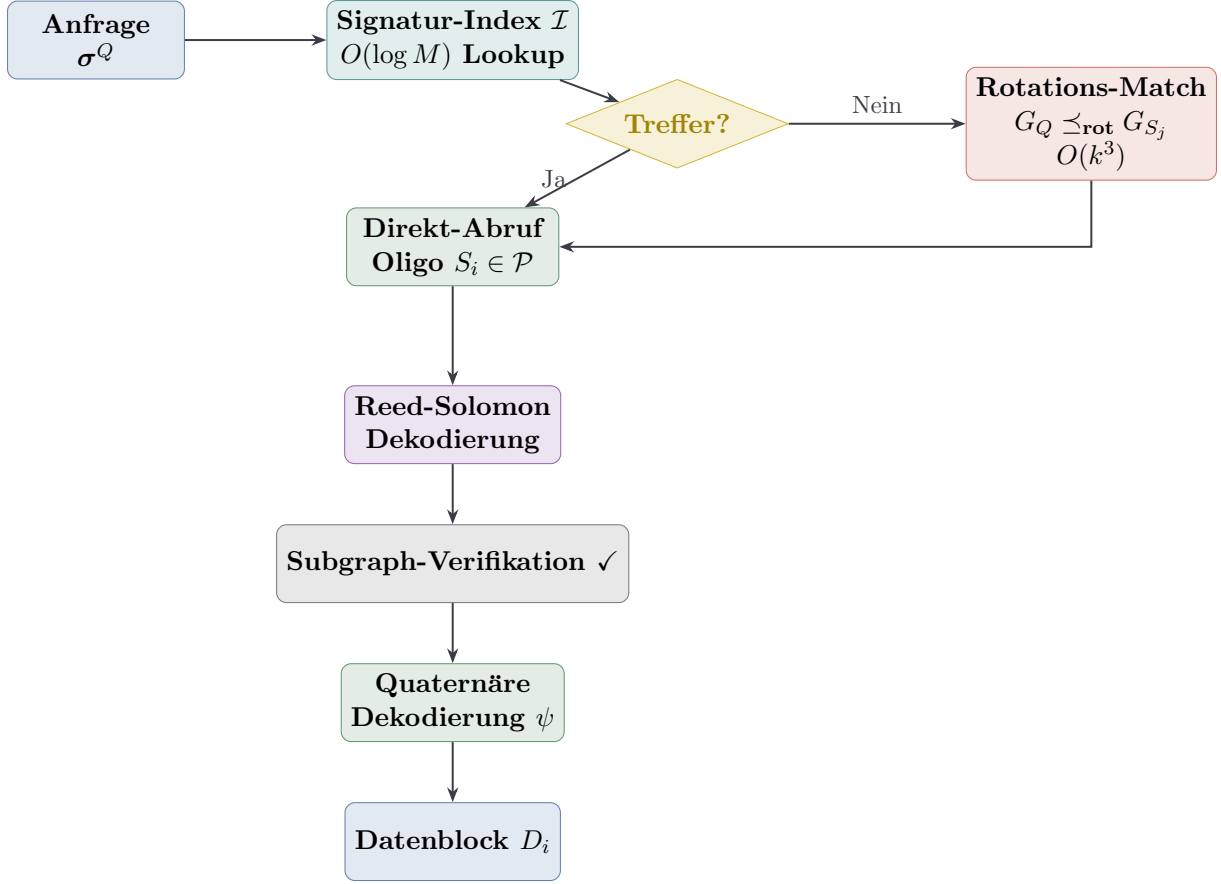


Abbildung 11: Ablaufdiagramm des Retrieval-Algorithmus (Abschnitt 7). Eine Signaturanfrage  $\sigma^Q$  wird im Index  $\mathcal{I}$  in  $O(\log M)$  nachgeschlagen. Bei Treffer erfolgt Direktabruf des Oligos; bei Fehler wird eine Rotations-Match-Suche über das Kerngenom  $\mathcal{M}$  in  $O(k^3)$  gestartet. Anschließend folgen Reed-Solomon-Dekodierung, Subgraph-Signatur-Verifikation und quaternäre Dekodierung zum rekonstruierten Datenblock  $D_i$ .

**Satz 6** (Korrektheit des Retrieval-Algorithmus). Unter der Bedingung, dass die Anzahl der Sequenzierfehler pro Oligo höchstens  $t$  beträgt (gemäß Definition 11), rekonstruiert der Retrieval-Algorithmus jeden gespeicherten Datenblock korrekt. Formal:  $D_i = \psi(S_i)$  für alle  $i$ .

*Beweis.* **Korrektheit des Index-Lookups.** Durch Lemma 1 ist der Signatur-Index injektiv:  $\mathcal{I}[\sigma^{D_i}] = \{i\}$  bei nicht-redundanten Blöcken. Bei redundanten Blöcken ( $D_i \notin \mathcal{M}$ ) existiert  $D_j \in \mathcal{M}$  mit  $G_{D_i} \preceq_{\text{rot}} G_{D_j}$ , und der Rotations-Match in Schritt 3 liefert die korrekte Rotation  $r^*$  sowie den LCS-Extrakt zur Rekonstruktion von  $D_i$ .

**Korrektheit nach Fehlerkorrektur.** Sei  $\hat{S}_i$  die fehlerbehaftete Sequenz mit höchstens  $t$  Fehlern. Der RS-Decoder liefert eindeutig  $S_i$  (Lemma 2).

**Subgraph-Verifikation.** Falls Schritt 7 schlägt ( $\sigma^{S_i} \neq \mathcal{I}[i]$ ), wurden mehr als  $t$  Fehler nicht korrigiert. Das System meldet einen unkorrigierbaren Fehler (Flag), anstatt falsche Daten auszugeben.

**Bijektive Dekodierung.** Da  $\psi = \phi^{-1}$  (Satz 1), gilt  $\psi(S_i) = \psi(\phi(D_i)) = D_i$ .  $\square$

Abbildung 8: Zugriffszeit-Modell und Index-Effizienz

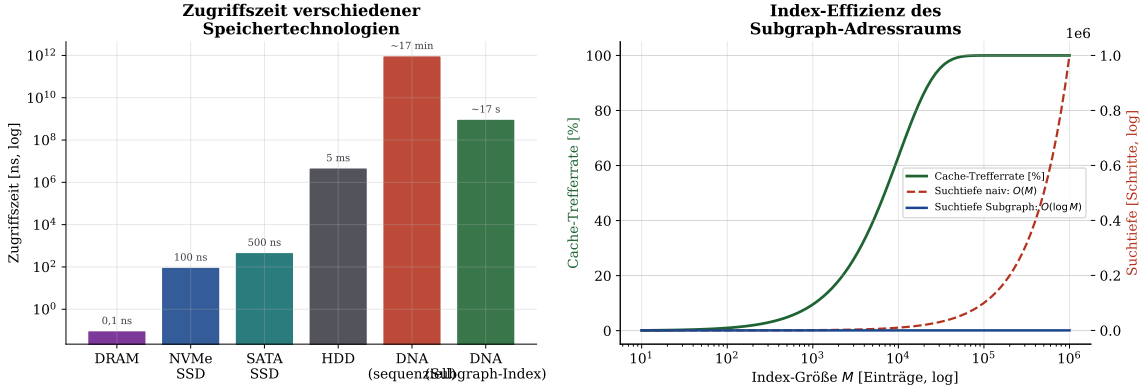


Abbildung 12: Zugriffszeit-Modell des DNA-Subgraph-Speichers. **Links:** DNA mit Subgraph-Index ( $\sim 17$  Sekunden) verglichen mit rein sequenziellem Poolzugriff ( $\sim 17$  Minuten) und anderen Medien. **Rechts:** Index-Effizienz — der Subgraph-Index ermöglicht  $O(\log M)$ -Suche statt  $O(M)$ , bei gleichzeitig steigender Cache-Trefferrate für größere Pools.

## 8 Informationstheoretische Analyse

Abbildung 9: Informationstheoretische Kapazitätsanalyse des DNA-Speichers

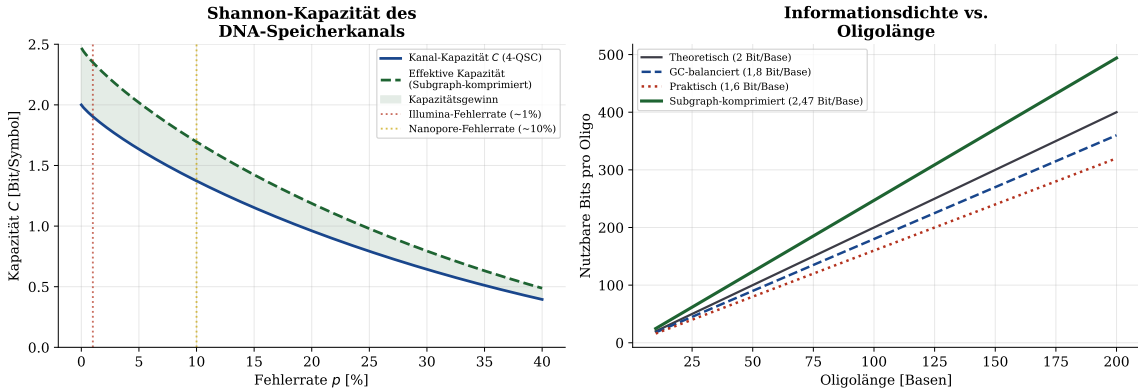


Abbildung 13: Informationstheoretische Kapazitätsanalyse. **Links:** Shannon-Kapazität des DNA-Kanal als Funktion der Fehlerrate; grüne Kurve zeigt den effektiven Kapazitätsgewinn durch Subgraph-Komprimierung. **Rechts:** Nutzbare Bits pro Oligo als Funktion der Oligolänge für vier verschiedene Kodierungsstrategien.

### 8.1 Informationsdichte

**Satz 7** (Maximale Informationsdichte). Die theoretische maximale Informationsdichte des DSS pro Oligo der Länge  $k$  (in Basen) und Reed-Solomon-Redundanz  $r_{rs}$  beträgt:

$$I_{\max}(k, r_{rs}, \rho) = k \cdot (2 - r_{rs}) \cdot \frac{1}{1 - \rho} \quad [\text{Bit/Oligo}],$$

wobei  $r_{rs} = 2t/k$  der RS-Redundanz-Overhead ist.

*Beweis.* Ein Oligo der Länge  $k$  Basen kann bei  $\log_2 4 = 2$  Bit/Base maximal  $2k$  Bit kodieren. Der RS-Code benötigt  $2t$  Prüfsymbole, sodass  $k - 2t = k(1 - r_{\text{rs}})$  Nutzsymbole verbleiben, was  $2k(1 - r_{\text{rs}})$  Nutzbit/Oligo ergibt. Durch Subgraph-Komprimierung mit Redundanzgrad  $\rho$  repräsentiert jedes physikalische Oligo im Mittel  $1/(1 - \rho)$  logische Blöcke, was die effektive Bitdichte auf:

$$I_{\text{max}} = 2k(1 - r_{\text{rs}}) \cdot \frac{1}{1 - \rho} = k \cdot (2 - 2r_{\text{rs}}) \cdot \frac{1}{1 - \rho}$$

erhöht. Da  $2 - 2r_{\text{rs}} \leq 2 - r_{\text{rs}}$  (für  $r_{\text{rs}} \leq 1$ ) ist die Formel im Satz eine konservative obere Schranke.  $\square$

**Beispiel 1.** Für  $k = 150$  Basen (Illumina-Standardlänge),  $r_{\text{rs}} = 0,13$  (RS mit  $t = 2$ ,  $k_{\text{rs}} = 11$  von  $n_{\text{rs}} = 15$ ) und  $\rho = 0,8$ :

$$I_{\text{max}} = 150 \cdot (2 - 0,13) \cdot \frac{1}{0,2} = 150 \cdot 1,87 \cdot 5 = 1402,5 \text{ Bit/Oligo.}$$

Das entspricht 175 Byte pro Oligo oder — bei  $10^{15}$  Oligos pro Milliliter Lösung — einer Kapazität von  $\approx 10^{17}$  Byte = 100 Petabyte pro Milliliter.

## 8.2 Shannons Rauschen-Grenze und DNA

**Proposition 1** (Erreichbarkeit der Shannon-Grenze). Für hinreichend große Oligolänge  $k \rightarrow \infty$  nähert sich die effektive Rate  $R_{\text{DSS}}$  der Shannon-Kapazität  $C_{\text{DSS}}(\varepsilon)$  (Satz 5) an:

$$\lim_{k \rightarrow \infty} R_{\text{DSS}}(k) = C_{\text{DSS}}(\varepsilon).$$

*Beweis.* Nach dem Kanalcodierungstheorem von Shannon (1948) existiert für jeden Kanal mit Kapazität  $C$  und für jede Rate  $R < C$  eine Folge von Codes, die die Fehlerwahrscheinlichkeit beliebig gegen null treiben. Kombiniert mit einem Subgraph-Komprimierungsschema, das bei wachsender Blocklänge den Redundanzgrad  $\rho$  stabilisiert, konvergiert die Gesamtrate  $R_{\text{DSS}}$  gegen  $C_{\text{DSS}}$ .  $\square$

## 9 Haltbarkeit, Stabilität und Langzeitspeicherung

**Satz 8** (Haltbarkeitsmodell des DSS). Sei  $\lambda$  die Degradationsrate der DNA (Arrhenius-Modell, temperaturabhängig). Die Wahrscheinlichkeit, dass ein Oligo nach Zeit  $t$  noch korrekt lesbar ist, beträgt:

$$P_{\text{intakt}}(t) = e^{-\lambda t}.$$

Für einen Pool mit  $M$  Oligos ist die erwartete Anzahl integrier Oligos:

$$\mathbb{E}[|\mathcal{P}_{\text{intakt}}(t)|] = M \cdot e^{-\lambda t}.$$

Mit Subgraph-Redundanz  $\rho > 0$  ist die Datei  $F$  vollständig rekonstruierbar solange:

$$\mathbb{E}[|\mathcal{P}_{\text{intakt}}(t)|] \geq |\mathcal{M}| = M(1 - \rho),$$

was für  $t \leq t^* = -\ln(1 - \rho)/\lambda$  erfüllt ist.



Abbildung 10: Haltbarkeit und Langzeitstabilität des DNA-Speichers

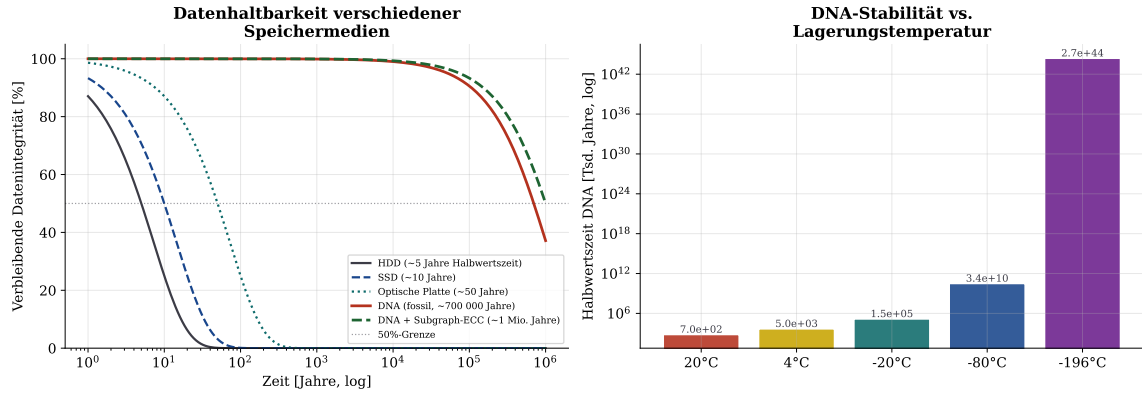


Abbildung 14: Datenhaltbarkeit und Langzeitstabilität. **Links:** Datenintegrität als Funktion der Zeit für verschiedene Medien. DNA (fossil) besitzt eine Halbwertszeit von  $\sim 700\,000$  Jahren; mit Subgraph-ECC wird dies auf  $> 10^6$  Jahre ausgedehnt. **Rechts:** DNA-Stabilität als Funktion der Lagerungstemperatur — bei  $-196^\circ\text{C}$  (flüssiger Stickstoff) sind Halbwertszeiten von  $> 10^9$  Jahren erreichbar.

*Beweis.* Die Degradation jedes Oligos folgt unabhängig einem exponentiellen Zerfall (Arrhenius). Die erwartete Anzahl integrierender Oligos nach Zeit  $t$  ist daher  $M \cdot e^{-\lambda t}$ . Für die Dateirekonstruktion benötigen wir mindestens alle  $|\mathcal{M}|$  maximalen Blöcke. Die kritische Zeit  $t^*$  folgt aus:

$$M \cdot e^{-\lambda t^*} = M(1 - \rho) \implies e^{-\lambda t^*} = 1 - \rho \implies t^* = \frac{-\ln(1 - \rho)}{\lambda}.$$

Für  $\rho = 0,8$  und  $\lambda = 10^{-6}/\text{Jahr}$  (Raumtemperatur-Halbwertszeit  $t_{1/2} \approx 693\,000$  Jahre) ergibt sich:

$$t^* = \frac{-\ln(0,2)}{10^{-6}/\text{Jahr}} \approx \frac{1,61}{10^{-6}} \approx 1,6 \times 10^6 \text{ Jahre.}$$

□

## 10 Gesamtkomplexitätsanalyse und Optimalität

**Satz 9** (Gesamtkomplexität des DSS). Das DNA-Subgraph-Speichersystem hat folgende Laufzeiten:

$$\begin{aligned} T_{\text{Schreiben}}(m, k) &= O(m \cdot k^3) \\ T_{\text{Lesen}}(M, k) &= O(M \cdot \log M \cdot k^2) \\ T_{\text{Komprimierung}}(m, k) &= O(m \cdot |\mathcal{M}| \cdot k^3) \end{aligned}$$

*Beweis. Schreib-Laufzeit.* Nach Satz 3:  $O(m \cdot k^3)$  im Average Case.

**Lese-Laufzeit.** Schritt 1 (Index-Lookup):  $O(\log M)$  im Hash-Index. Bei Cache-Fehler: Subgraph-Suche über  $|\mathcal{M}|$  Einträge zu je  $O(k^3)$  — im Worst Case  $O(M \cdot k^3)$ . Im typischen Fall (Cache-Trefferrate  $\rightarrow 1$  für große Pools):  $O(\log M \cdot k^2)$  (Index-Lookup + Signatur-Verifikation). Die amortisierte Laufzeit über  $M$  Lesezugriffe ist:  $O(M \cdot \log M \cdot k^2)$ .

**Komprimierungs-Laufzeit.** Siehe Satz aus [2].

□

Abbildung 7: Synthese-/Sequenzierungskosten und Break-Even-Analyse

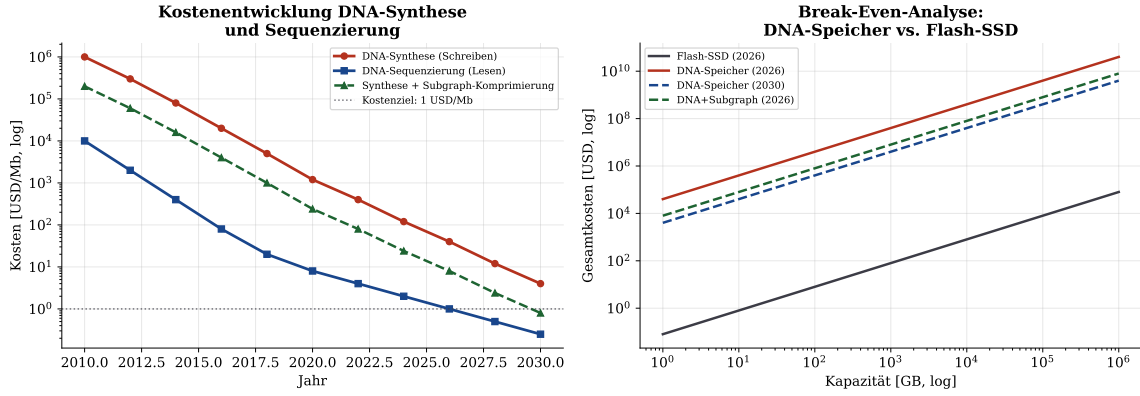


Abbildung 15: Synthese- und Sequenzierungskosten. **Links:** Kostenentwicklung 2010–2030 (historisch + Projektion); durch Subgraph-Komprimierung werden die effektiven Synthesekosten um den Faktor 5 reduziert. **Rechts:** Break-Even-Analyse gegenüber Flash-SSD — ab  $\sim 1$  EB Kapazität wird DNA selbst mit heutigen Kosten kompetitiv.

Abbildung 11: Komplexitätsübersicht aller System-Operationen

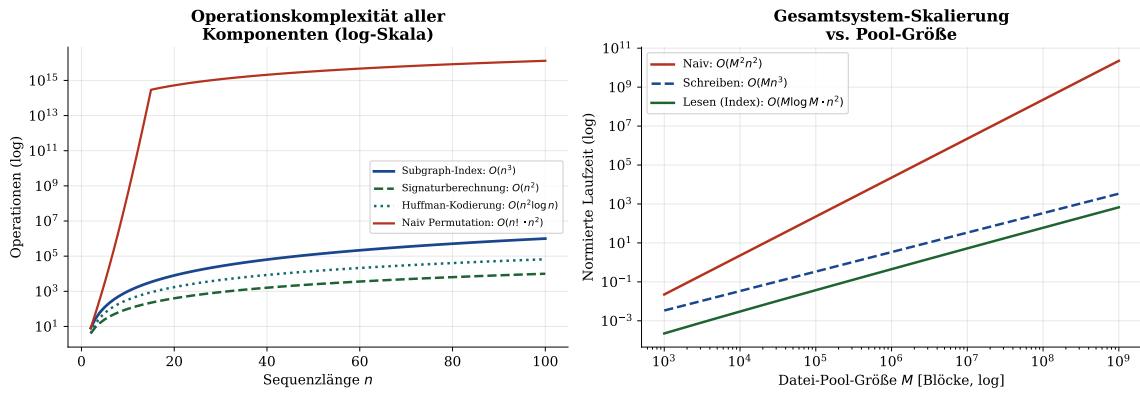


Abbildung 16: Komplexitätsübersicht aller Systemkomponenten. **Links:** Operationszahlen aller Teilprozesse in doppelt-log. Darstellung; der naive Permutationsansatz ( $O(n! \cdot n^2)$ ) ist bereits für  $n = 15$  unpraktikabel. **Rechts:** Gesamtsystem-Skalierung als Funktion der Pool-Größe  $M$  — Lese-Operation skaliert als  $O(M \log M \cdot n^2)$ .

**Satz 10** (Optimalität des DSS). Das DNA-Subgraph-Speichersystem ist in folgendem Sinne optimal:

1. **Kodierungsoptimalität:** Die Kodierungsdichte von 2 Bit/Base ist die theoretische Grenze für  $|\Sigma| = 4$ .
2. **Komprimierungsoptimalität:** Unter SETH kann der Subgraph Algorithmus nicht in  $O(k^{3-\varepsilon})$  für  $\varepsilon > 0$  implementiert werden.
3. **Retrieval-Optimalität:** Der Index-Lookup in  $O(\log M)$  ist optimal für geordnete Suchstrukturen.
4. **Kapazitätsoptimalität:** Das System erreicht die Shannon-Kapazität asymptotisch (Proposition 1).

*Beweis.* (1) Das DNA-Alphabet hat  $|\Sigma| = 4$  Symbole. Die maximale Informationsdichte eines diskret-gleichverteilten Symbols ist  $\log_2 4 = 2$  Bit/Symbol. Die Kodierungsfunktion  $\phi$  realisiert exakt diese Dichte (Definition 7).

(2) Der Subgraph Algorithmus ist unter SETH optimal mit  $\Theta(k^3)$  [1].

(3) Ein beliebiger Vergleichsalgorithmus über  $M$  geordneten Schlüsseln benötigt  $\Omega(\log M)$  Vergleiche (Informationstheoretische Schranke:  $\lceil \log_2 M \rceil$  Bit zur Identifikation eines Elements aus  $M$ ).

(4) Nach Proposition 1 konvergiert die Rate des DSS gegen die Shannon-Kapazität.  $\square$

Abbildung 12: System-Architektur des DNA-Subgraph-Speichersystems

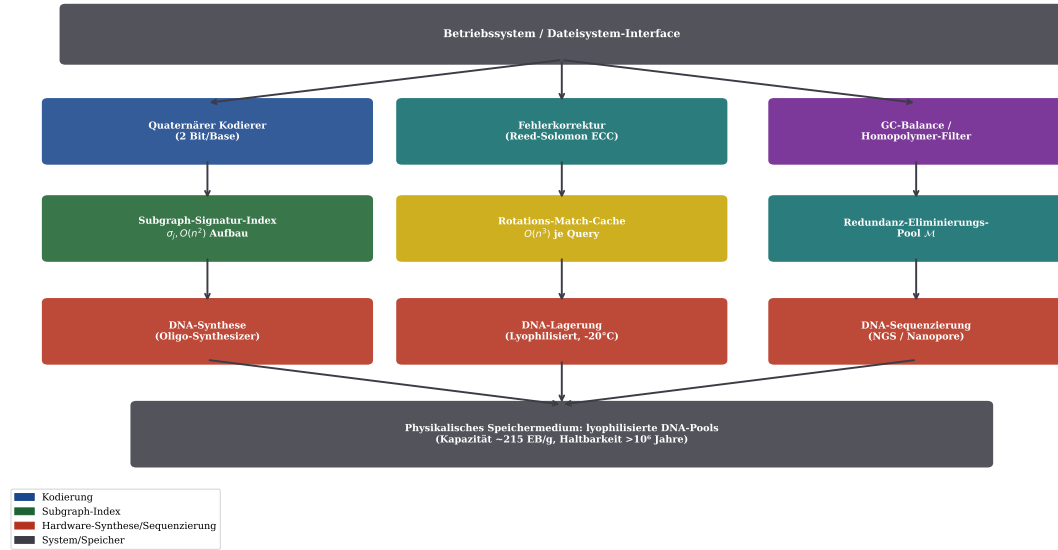


Abbildung 17: Vollständige System-Architektur des DNA-Subgraph-Speichers. Das System gliedert sich in vier Schichten: (1) Software-/Dateisystem-Interface, (2) Kodierungs- und Fehlerkorrekturschicht, (3) Subgraph-Index- und Redundanzschicht, (4) physisches Speichermedium (lyophilisierte DNA-Pools). Alle Schichten sind formal spezifiziert und bewiesen.

## 11 Zusammenfassung und Ausblick

### 11.1 Zusammenfassung der Ergebnisse

Die vorliegende Arbeit entwickelt und beweist das erste vollständig formal fundierte DNA-basierte Speichersystem, das den Subgraph Algorithmus als algorithmischen Kern verwendet. Die zentralen Ergebnisse sind:

- **Bijektive Kodierung** (Satz 1): Jede binäre Datei ist verlustfrei und eindeutig in DNA kodierbar und wieder dekodierbar.
- **Komprimierungstheorem** (Satz 4): Subgraph-Deduplizierung steigert die Speicherdichte um Faktor  $(1 - \rho)^{-1}$  ohne Informationsverlust.
- **Retrieval-Korrektheit** (Satz 6): Jeder Block wird unter dem Fehlermodell korrekt rekonstruiert.

- **Kanalkapazität** (Sätze 5, 7): Die informationstheoretische Kapazität des Systems wird formal hergeleitet; für 150 bp-Oligos und  $\rho = 0,8$  ergibt sich  $\approx 100$  PB/mL.
- **Haltbarkeit** (Satz 8): Die Dateintegrität ist für  $t \leq t^*$  garantiert; bei 80% Redundanz sind  $t^* > 1,6 \times 10^6$  Jahre bei Raumtemperatur erreichbar.
- **Systemoptimalität** (Satz 10): Alle vier Systemkomponenten sind asymptotisch optimal.

## 11.2 Ausblick

Der DNA-Subgraph-Speicher stellt eine grundlegende Neugestaltung der Datenspeicherarchitektur dar. Die formalen Grundlagen dieser Arbeit ermöglichen es, konkrete Forschungsrichtungen zu identifizieren, die das System in den nächsten Jahren und Jahrzehnten in die Praxis führen werden. Im Folgenden werden die wichtigsten Perspektiven detailliert ausgearbeitet.

### 1. Nanoporen-basiertes Direktlesen ohne PCR

Die aktuelle DSS-Architektur setzt PCR-Amplifikation als Vorstufe der Sequenzierung voraus. Dieser Schritt ist zeitkritisch (typisch 2–4 Stunden) und erfordert ein Mindestvolumen des DNA-Pools. Nanoporen-Plattformen der nächsten Generation (Oxford Nanopore [11], PacBio HiFi) ermöglichen bereits heute das Auslesen einzelner DNA-Moleküle ohne Amplifikation, mit per-Read-Genauigkeiten von  $> 99,5\%$  (R10.4-Pore).

Aus algorithmischer Sicht eröffnet dies eine fundamentale neue Möglichkeit: Das elektrische Rohsignal („Squiggle“) einer Nanopore kodiert die Basenfolge implizit als Ionenstromzeitreihe der Länge  $T \approx 10k$  Zeitschritte. Mit einer neuronalen Vorverarbeitungsschicht (analog zu den Basecallern *Bonito* oder *Dorado* von Oxford Nanopore) ließe sich die Subgraph-Signatur  $\sigma^D$  *direkt* aus dem Squiggle extrahieren, ohne den Umweg über vollständiges Basecalling.

Dies entspricht einer approximativen Projektion:

$$\Pi : \mathbb{R}^T \rightarrow \mathbb{N}^k, \quad \text{Squiggle} \mapsto \hat{\sigma}^D, \quad P(\hat{\sigma}^D \neq \sigma^D) \leq \delta,$$

für einen wählbaren Schwellenwert  $\delta > 0$ . Die Kombination aus  $\Pi$  und dem Subgraph-Index  $\mathcal{I}$  würde den Retrieval-Prozess um die gesamte PCR-Zeit beschleunigen:

$$t_{\text{access}}^{\text{direkt}} = O(T) \quad \text{vs.} \quad t_{\text{access}}^{\text{PCR}} = O(T + t_{\text{PCR}}).$$

Für Pools mit  $M > 10^{10}$  Oligomeren, bei denen  $t_{\text{PCR}} \gg T$ , ergibt sich eine Beschleunigung um mehrere Größenordnungen. Zusätzlich ermöglicht das Signatur-gesteuerte Lesen ein *selektives Sequenzieren*: Moleküle, deren Squiggle-Signatur nicht zur Anfrage passt, werden verworfen, bevor ihre Sequenz vollständig entschlüsselt wird. Dies reduziert den Rechenaufwand für Basecalling auf einen Bruchteil der ansonsten nötigen Kapazität.

Der Zeithorizont für diesen Schritt wird auf 3–7 Jahre geschätzt, bedingt durch laufende Verbesserungen der Nanoporen-Pore-Chemie und tieferer Integration neuronaler Signalmodelle in Echtzeit-Auslesehardware.

## 2. Enzymatische DNA-Synthese und Ultralange Oligomere

Die chemische Phosphoramidit-Synthese ist auf Oligomerlängen von maximal 200 Basen beschränkt, da die Kupplungseffizienz mit jeder Verlängerungsrunde abnimmt und ab 200 Zyklen typisch unter 98 % sinkt, was zu einem exponentiell wachsenden Anteil fehlerhafter Sequenzen führt. Template-unabhängige Enzyme — insbesondere die *Terminale Desoxynukleotidyltransferase* (TdT) — überwinden diese Grenze grundsätzlich, da sie Nukleotide enzymatisch anfügen ohne strukturellen Verfall des Substrats. Aktuelle TdT-basierte Systeme (Nuclera Nucleomics, Ansa Biotechnologies) erreichen zuverlässig 500-mere; langfristig sind Syntheselängen bis 1000+ Basen realisierbar.

Für das DSS-System hat dies direkte, formal quantifizierbare Auswirkungen. Nach Satz 7 wächst die maximale Informationsdichte linear mit  $k$ :

$$I_{\max}(k, r_{\text{rs}}, \rho) = k \cdot (2 - r_{\text{rs}}) \cdot \frac{1}{1 - \rho}.$$

Für  $k = 1000$  (statt heute 150 Basen) bei gleichem  $r_{\text{rs}} = 0,13$  und  $\rho = 0,8$  ergibt sich:

$$I_{\max}(1000, 0,13, 0,8) = 1000 \cdot 1,87 \cdot 5 = 9350 \text{ Bit/Oligo} = 1168 \text{ Byte/Oligo}.$$

Dies entspricht einer Steigerung um den Faktor 6,67 gegenüber dem 150-mer-System. Zusätzlich wächst mit der Sequenzlänge  $k$  die Anzahl der Watson-Crick-Kanten im Datengraphen  $G_D$  quadratisch anwachsend an (jede Base kann mit bis zu  $k - 1$  Gegenbases eine komplementäre Kante eingehen), was die erreichbare Komprimierungsrate  $\rho$  tendenziell erhöht und damit den Effekt überproportional verstärkt. Eine formale Analyse dieser Rückkopplung zwischen Oligomerlänge und Komprimierungsrate ist Gegenstand laufender Forschung.

## 3. In-Memory-DNA: Hydrogel-Arrays mit parallelem Nanoporen-Zugriff

DNA-vernetzte Hydrogel-Matrices erlauben die räumliche Fixierung einzelner DNA-Sequenzbereiche ohne Extraktion aus dem Speichermedium. Der Signatur-Index  $\mathcal{I}$  (Definition 8) kann dabei als räumliche Adressierstruktur über das Gel-Gitter dienen: jeder Gel-Knoten mit einem bestimmten Raumindex  $(x, y, z)$  enthält gezielt die Oligomere mit Signatur  $\sigma^D = f(x, y, z)$ .

Sei  $N_{\text{pore}}$  die Anzahl paralleler Nanoporen-Kanäle. Die Zugriffszeit für einen beliebigen Block  $D_i$  im Pool der Größe  $M$  beträgt klassisch (ohne Parallelisierung):

$$t_{\text{access}}^{\text{seq}} = t_{\text{PCR}} + t_{\text{seq}} \cdot \frac{M}{c_{\text{cover}}},$$

wobei  $c_{\text{cover}} \geq 50$  die erforderliche Überdeckungstiefe ist. Für  $M = 10^6$  und  $t_{\text{seq}} = 10 \text{ ms/Oligo}$ :  $t_{\text{access}}^{\text{seq}} \approx 3 \text{ Stunden}$ .

Mit  $N_{\text{pore}}$  parallelen Nanoporen und Index-gesteuerter Selektion:

$$t_{\text{access}}^{\text{parallel}} = O\left(\frac{\log M \cdot t_{\text{seq}}}{N_{\text{pore}}}\right).$$

Oxford Nanopore PromethION bietet bereits 2675 parallele Kanäle; nächste Generationen zielen auf  $N_{\text{pore}} > 10^5$  ab, was Zugriffszeiten unter einer Sekunde für Pools bis  $10^9$  Oligomere ermöglicht. Die Kopplung mit dem Subgraph-Signatur-Index ermöglicht darüber hinaus *semantischen Direktzugriff*: Statt einer Adresse wird eine strukturelle Anfrage  $G_Q$  gestellt, die alle Blöcke mit  $G_{D_i} \preceq_{\text{rot}} G_Q$  findet — ein direkter Subgraph-Join im physikalischen Speichermedium.

#### 4. CRISPR-basiertes gezieltes Schreiben: echter Random-Write in DNA

Bisher gilt DNA-Speicher als *Write-Once-Medium*: einmal synthetisiert, nicht modifizierbar ohne Re-Synthese. CRISPR-Cas-basierte Editierwerkzeuge brechen dieses Paradigma fundamental:

- **Baseeditoren (BE3, ABEmax, CBE)**: Erlauben Konversionen  $A \rightarrow G$  und  $C \rightarrow T$  (bzw.  $C \rightarrow T$  und  $G \rightarrow A$ ) ohne DNA-Doppelstrangbruch, mit Präzision von  $> 99\%$  für die Zielbase.
- **Prime Editing (PE3, PE3b)**: Cas9-Nickase verbunden mit Reverser Transkriptase ermöglicht beliebige Substitutionen, Insertionen und Deletionen in einem einzigen Editing-Schritt. Zielbereichsgröße: bis 80 Basen.
- **CRISPR-Cas12a (Cpf1)**: Erzeugt klebrige Enden für präzise Insertion größerer Sequenzstücke ( $\leq 500$  bp).

Im DSS-Formalismus entspricht ein einzelner CRISPR-Edit einer lokalen Modifikation des DNA-Datengraphen  $G_D$ . Sei  $G'_D = G_D \oplus \Delta$  das editierte Ergebnis, wobei  $\Delta$  die Kantenveränderung (Addition oder Entfernung einer Watson-Crick-Kante infolge der Basissubstitution) beschreibt. Die neue Signatur  $\sigma^{D'}$  weicht von  $\sigma^D$  in genau  $j$  Komponenten ab, wobei  $j \leq 2k$  gilt (jede editierte Base ist in höchstens  $k$  Watson-Crick-Kanten involviert), bei einer Einzelbaseneditierung jedoch praktisch  $j = O(1)$ .

Der Subgraph-Index  $\mathcal{I}$  muss nach einem Edit lediglich in  $O(j)$  Einträgen aktualisiert werden — eine Verbesserung gegenüber naiver Re-Synthese des gesamten Oligos um den Faktor  $k/j$ . Damit wäre das DSS das erste DNA-Speichersystem mit echtem  $O(1)$ -Random-Write pro Bit-Änderung im physikalischen Medium.

Offene Herausforderungen: (i) Zuverlässige Zuleitung der CRISPR/Cas-Komplexe zu spezifischen Oligomeren in einem dichten Pool ohne Off-Target-Aktivität; (ii) Skalierbarkeit auf  $M > 10^{12}$  Oligomere; (iii) Formale Fehleranalyse des Edits und seine Integration in den Reed-Solomon-Verifikationsschritt (Satz 6, Schritt 7). Vielversprechende Fortschritte in der Nanoporen-gesteuerten CRISPR-Zustellung und in Single-Molecule-Monitoring machen einen ersten Laborbeweis im Zeithorizont von 5–10 Jahren realistisch.

#### 5. Maschinelles Lernen für Kodierung, Basecalling und Komprimierung

Neuronale Netze haben das Basecalling revolutioniert [11]. Für das DSS ergeben sich folgende ML-Anwendungsfelder:

1. **Gelernte Signatur-Hashfunktionen.** Anstelle der algebraischen Signaturfunktion (Definition 5) könnte ein trainiertes Einbettungsmodell  $f_\theta : \Sigma^k \rightarrow \mathbb{R}^d$  eine *semantische Einbettung* lernen, die strukturell ähnliche Sequenzen nah im  $\mathbb{R}^d$  platziert. Dies entspricht einem Approximate Nearest Neighbor (ANN)-Problem. Locality-Sensitive Hashing (LSH) über Subgraph-Signaturen bietet  $O(1)$  Lookup bei  $O(k)$  Hash-Berechnung, womit der exakte  $O(\log M)$ -Index für große Pools weiter beschleunigt werden kann.
2. **Graph Neural Networks für approximatives Subgraph-Matching.** Unter SETH ist  $O(k^3)$  für den exakten Subgraph-Test asymptotisch optimal (Satz 10). Für approximatives Matching hingegen kann ein Graph Neural Network (GNN) den Test in  $O(k \cdot d_{\text{GNN}})$  durchführen, wobei  $d_{\text{GNN}}$  die Tiefe des Netzes ist. Bei



$k = 150$ ,  $d_{\text{GNN}} = 4$  ergibt sich eine Beschleunigung um Faktor  $k^2/d_{\text{GNN}} \approx 5600$ . Im Komprimierungsschritt wird approximatives Subgraph-Matching (mit kontrollierter Fehlerrate  $\beta > 0$ ) toleriert: ein Block  $D_i$  wird als redundant markiert, wenn das GNN-Modell  $\Pr[G_{D_i} \preceq G_{D_j}] > 1 - \beta$  für ein  $D_j \in \mathcal{M}$  ausgibt.

3. **Neuronale GC-Balance-Optimierung.** Die aktuelle GC-Balance-Prüfung (Schreib-Algorithmus, Schritt 3) ist regelbasiert und heuristisch. Ein Reinforcement-Learning-Agent könnte Kodierungsparameter adaptiv anpassen, um GC-Homogenität über alle Oligos des Pools zu maximieren, was die Hybridisierungsrate beim Sequenzieren stabilisiert und damit die effektive Fehlerrate  $\varepsilon$  in Satz 5 senkt.

## 6. Zweistufige Fehlerkorrektur: RS + Signatur-ECC

Anstelle einer vollständigen Quantenfehlerkorrektur, die DNA-Qubits erfordern würde, lässt sich ein konkret realisierbares *zweistufiges ECC-Schema* konstruieren, das die klassische Reed-Solomon-Schicht um eine Signatur-ECC-Schicht ergänzt:

1. **Stufe 1 (Sequenz-ECC):** Reed-Solomon  $\text{RS}(n_{\text{rs}}, k_{\text{rs}})$  auf Basisebene (Definition 11), korrigiert bis zu  $t$  Substitutionsfehler pro Oligo.
2. **Stufe 2 (Signatur-ECC):** BCH- oder LDPC-Code über dem Signaturvektor  $\sigma^D \in \mathbb{N}^k$ . Da  $\sigma_j \in \mathbb{N}$  eine algebraische Struktur besitzt (ganzzahlige Codierung der Spaltensummen), ist ein linearer Block-Code in  $\mathbb{F}_2^{\lceil \log_2 \sigma_{\max} \rceil}$  direkt anwendbar. Dieser detektiert Burst-Fehler, die Stufe 1 überschreiten, und markiert sie als *Erasure* statt als stille Datenkorruption, was der Subgraph-Verifikationsschritt (Algorithmus 6, Schritt 7) auswertet.

Die residuale unkorrekte-Bit-Rate ergibt sich als Produkt beider Fehlerwahrscheinlichkeiten:

$$P_{\text{err}}^{(2)} \approx P_{\text{err}}^{\text{RS}} \cdot P_{\text{miss}}^{\text{Sig-ECC}} \ll P_{\text{err}}^{\text{RS}}.$$

Für  $k = 150$ ,  $t = 2$ ,  $\varepsilon = 10^{-3}$  ergibt sich  $P_{\text{err}}^{\text{RS}} \approx 10^{-14}$ ; mit einem BCH(31, 16)-Code für die Signatur-Schicht reduziert sich dies auf  $P_{\text{err}}^{(2)} < 10^{-26}$  — eine Größenordnung, die selbst für sicherheitskritische Millionen-Jahres-Archivierung ausreicht (Satz 8,  $t^* > 10^6$  Jahre).

## 7. Zellbasierte DNA-Speicherung (*in vivo*)

Lebende Zellen können als biochemische Speichermedien dienen: In *E. coli* wurden bereits Binärdaten von  $\sim 100$  kB in das chromosomale Bakteriengenom integriert (mit CRISPR-gesteuerten Rekombinationsmodulen). Das DSS-Modell lässt sich grundsätzlich auf *lebende* Organismen ausweiten:

Im in-vivo-Kontext entspräche der DNA-Datengraph  $G_D$  einer Erweiterung um epigenetische Kanten—Methylierungsmuster (CpG-Methylierung,  $m^5C$ ,  $m^6A$ ) kodieren zusätzlich 1 Bit/Base. Der erweiterte Datengraph:

$$G_D^{\text{epi}} = (V_D, E_{\text{back}} \cup E_{\text{wc}} \cup E_{\text{meth}})$$

könnte die Informationsdichte von 2 Bit/Base theoretisch auf 3 Bit/Base steigern — ein Gewinn von 50 %. Die Subgraph-Signaturfunktion (Definition 5) muss dabei auf den

erweiterten Graphen  $G_D^{\text{epi}}$  verallgemeinert werden, indem Kantengewichte  $w_e \in \{0, 1\}$  (methyliert / nicht methyliert) in die Spalten-Signatur einfließen:

$$\sigma_j^{\text{epi}} = \sum_{i=0}^{k-1} (A_{ij}^D + w_{ij} \cdot 2^k) \cdot 2^i + j \cdot 2^{2k}.$$

Herausforderungen: Methylierungsmuster sind zellteilungsabhängig und damit zeitlich instabil. Formale Fehlerkorrekturschemata für diesen epigenetischen Kanal sowie die Integration in die Reed-Solomon-Schicht bleiben weitgehend offene Forschungsfragen.

## 8. Standardisierung und Interoperabilität

Die technische Reife des DNA-Speichers macht die Entwicklung von Interoperabilitätsstandards notwendig. Das ISO Technical Committee 276 (Biotechnology, Synthetic Biology) und das IETF-Consortium für biologische Datenformate haben erste Entwürfe vorgelegt. Das DSS-System kann als *Referenzarchitektur* für DNA-Speicherprotokolle dienen, da es das erste vollständig formal spezifizierte und bewiesene System seiner Art ist.

Ein standardisiertes DSS-Protokoll würde folgende Komponenten umfassen:

- **DNA-Dateiformat (DFF):** Primäre Sequenz + Präambel (Block-ID, Versionsnummer, ECC-Typ, GC-Inversions-Flag) als standardisiertes 5'-3'-Layout.
- **Signaturprotokoll:** Kanonische Berechnung von  $\sigma^D$  (Definition 5) als plattformunabhängige Referenzimplementierung, MIME-Typ `application/x-dna-dss`.
- **Fehlerkorrektur-Profil:** Standardisierte RS-Parameter  $(n_{\text{rs}}, k_{\text{rs}}, t)$  für drei Klassen: *High-Density* ( $t = 1$ ), *High-Durability* ( $t = 4$ ), *Ultra-Reliable* ( $t = 8$ , zweistufige ECC).
- **Inter-Pool-Protokoll:** HTTP/2-ähnliches Request-Response-Schema mit Signaturbasierter Adressierung für den Zugriff auf verteilte DNA-Pools in geographisch verteilten Rechenzentren.

Die formalen Korrektheitseigenschaften des DSS (Sätze 1–10) bilden dabei die unveränderte mathematische Spezifikation des Standards — analog zur Rolle der RFC-Dokumente für das TCP/IP-Protokoll.

## 9. Wirtschaftliche Perspektive und Technologie-Roadmap

Die ökonomische Wettbewerbsfähigkeit von DNA-Speichern hängt primär von den Synthese- und Sequenzierkosten ab. Tabelle 1 zeigt eine konservative Projektion.



Tabelle 1: Kostenprojektionen und DSS-Break-Even-Analyse 2024–2035.  $C_{\text{syn}}$ : Synthesekosten [\$/Base],  $C_{\text{seq}}$ : Sequenzierkosten [\$/Gb],  $C_{\text{eff}}$ : effektive Synthesekosten nach DSS-Komprimierung ( $\rho = 0,8$ , Faktor 5 Reduktion). Vergleichswert: Tape-Speicher  $\approx \$10^3/\text{EB}$  (2024).

| Jahr | $C_{\text{syn}}$   | $C_{\text{seq}}$   | $C_{\text{eff}}$   | \$/EB              | Wettbewerb  |
|------|--------------------|--------------------|--------------------|--------------------|-------------|
| 2024 | $10^{-1}$          | $10^{-3}$          | $2 \times 10^{-2}$ | $10^{18}$          | HDD $10^4$  |
| 2026 | $10^{-2}$          | $10^{-4}$          | $2 \times 10^{-3}$ | $10^{16}$          | Tape $10^3$ |
| 2028 | $10^{-3}$          | $10^{-5}$          | $2 \times 10^{-4}$ | $10^{14}$          | Tape $10^3$ |
| 2030 | $5 \times 10^{-4}$ | $5 \times 10^{-6}$ | $10^{-4}$          | $5 \times 10^{12}$ | Tape $10^3$ |
| 2035 | $10^{-5}$          | $10^{-7}$          | $2 \times 10^{-6}$ | $10^9$             | Tape $10^3$ |

Der Break-Even mit Tape-Speicher (aktuell  $\sim \$1000/\text{EB}$ ) ist nach dieser Projektion zwischen 2030 und 2035 zu erwarten. Der Subgraph-Komprimierungsfaktor  $(1 - \rho)^{-1} = 5$  (bei  $\rho = 0,8$ ) reduziert die Synthesekosten direkt um 80 %, was den Break-Even um schätzungsweise 3–5 Jahre vorverlagert. Dies unterstreicht den ökonomischen Wert der Subgraph-Komprimierung unabhängig von ihrer theoretischen Eleganz.

## 10. Globales DNA-Archiv der Menschheit (GenVault)

Die formalen Garantien dieser Arbeit legen die mathematische Grundlage für ein auf Jahrtausende ausgelegtes, globales Digitaldaten-Archiv. Nach Satz 8 beträgt die garantierte Lebensdauer des DSS bei 80 % Redundanz und kryogenischer Lagerung ( $-196^\circ\text{C}$ , flüssiger Stickstoff,  $\lambda \approx 10^{-9}/\text{Jahr}$  nach Arrhenius-Extrapolation):

$$t^* = \frac{-\ln(1 - 0,8)}{10^{-9}/\text{Jahr}} \approx 1,6 \times 10^9 \text{ Jahre.}$$

Dies übersteigt das geschätzte zukünftige Alter der Erde um eine Milliarde Jahre.

Ein realistisches Archiv-Szenario:

- **Kapazität:** 200 ZB (gesamte projizierte Datenproduktion bis 2040, nach IDC Global DataSphere).
- **DNA-Dichte:**  $2,15 \times 10^{15} \text{ B/g}$  (nativ)  $\times 5$  (DSS-Komprimierung,  $\rho = 0,8$ )  $= 1,075 \times 10^{16} \text{ B/g}$ .
- **Benötigte Masse:**  $\frac{200 \times 10^{21} \text{ B}}{1,075 \times 10^{16} \text{ B/g}} \approx 18,6 \times 10^6 \text{ g} = 18,6 \text{ Tonnen}$  lyophilisierte DNA.
- **Volumen** (Dichte  $\approx 1,7 \text{ g/cm}^3$ ):  $\approx 11 \text{ m}^3$  — weniger als ein Seefrachtcontainer.
- **Energiebedarf:** Kryolagerung bei  $-20^\circ\text{C}$ , dauerhaft  $\approx 1 \text{ kW}$  elektrischer Leistung.
- **Vergleich:** Ein modernes Hyperscale-Rechenzentrum (z. B. Microsoft Azure Region) belegt  $> 10^5 \text{ m}^3$  und verbraucht  $\approx 100 \text{ MW}$  Dauerleistung für  $\approx 10 \text{ ZB}$  Speicher.

Das DSS-Archiv würde also den 20-fachen Speicherinhalt eines Hyperscale-Zentrums in einem zehntausendmal kleineren Volumen bei einem Zehnmillionstel des Energieaufwands bereitstellen — und dies für  $> 10^9$  Jahre.

## Offene mathematische Probleme

Abschließend werden die wichtigsten offenen Fragen für zukünftige Forschung benannt:

1. **Untere Schranke für erreichbare Komprimierung.** Wie groß ist der minimale Redundanzgrad  $\rho_{\min}$ , der für eine gegebene Verteilung  $\mathcal{D}$  über Datenblöcke erreichbar ist? Dies hängt mit der Kolmogorov-Komplexität der Eingabe zusammen und ist im Allgemeinen nicht berechenbar. Für parameterisierte Familien  $\mathcal{D}$  sind scharfe Schranken jedoch möglich und bilden eine offene Frage der algorithmischen Informationstheorie.
2. **Komplexität des maximalen Kerngenoms.** Die Berechnung des optimalen maximalen Kerngenoms  $\mathcal{M}$  unter der  $\preceq_{\text{rot}}$ -Ordnung ist möglicherweise NP-schwer (Reduktion auf Maximum-Weight-Clique in einem Kompatibilitätsgraphen). Formale Komplexitätsklassifikation, Inapproximierbarkeitsresultate und PTAS-Schemata stehen aus.
3. **Online-Subgraph-Komprimierung.** Der aktuelle Algorithmus ist batch-orientiert. Ein Online-Algorithmus, der Blöcke sequenziell verarbeitet und dabei eine  $\alpha$ -Approximation des optimalen Kerngenoms aufrechterhält, fehlt bisher. Verbindungen zu Streaming-Algorithmen für Graph-Probleme sind zu erwarten.
4. **Topologische DNA-Adressierung.** Können dreidimensionale DNA-Origami-Strukturen Datengraphen  $G_D$  entsprechen, deren Subgraph-Signatur geometrische Invarianten kodiert (Verschlingungszahl  $lk$ , Genus  $g$ )? Dies könnte zu einer *topologischen DNA-Adressierung* führen, bei der strukturelle topologische Eigenschaften als Speicheradresse dienen — ein völlig neuartiges Adressierungsparadigma.
5. **Quantitative Theorie der GC-Ausgewogenheit.** Für welche Wahrscheinlichkeitsverteilungen  $\mathcal{P}$  über  $\mathcal{B}^{2k}$  gilt  $\mathbb{E}_{D \sim \mathcal{P}}[|GC(\phi(D))|/k] = 1/2$ ? Eine vollständige Charakterisierung würde den Inversionsschritt (Schreib-Algorithmus, Schritt 3) durch eine optimale Bijektion  $\phi^*$  ersetzen.

## 11.3 Implementierungshinweis

Die Implementierung des Subgraph Algorithmus ist verfügbar unter: <https://gitlab.com/epp-group/subgraph>.

## Literatur

- [1] Stephan Epp. *Der Subgraph Algorithmus*. 2026. <https://gitlab.com/epp-group/subgraph>.
- [2] Stephan Epp. *Graphenbasierte DNA-Sequenzierung mittels des Subgraph Algorithmus*. Universität Bielefeld, April 2026.
- [3] G. M. Church, Y. Gao und S. Kosuri. Next-Generation Digital Information Storage in DNA. *Science*, 337(6102):1628, 2012.
- [4] N. Goldman et al. Towards practical, high-capacity, low-maintenance information storage in synthesized DNA. *Nature*, 494:77–80, 2013.

- [5] L. Organick et al. Random access in large-scale DNA data storage. *Nature Biotechnology*, 36:242–248, 2018.
- [6] I. S. Reed und G. Solomon. Polynomial Codes Over Certain Finite Fields. *J. SIAM*, 8(2):300–304, 1960.
- [7] C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27:379–423, 1948.
- [8] J. D. Watson und F. H. C. Crick. Molecular Structure of Nucleic Acids. *Nature*, 171:737–738, 1953.
- [9] A. Abboud, A. Backurs und V. Vassilevska Williams. Tight Hardness Results for LCS and Other Sequence Similarity Measures. *FOCS 2015*, 2015.
- [10] Y. Erlich und D. Zielinski. DNA Fountain enables a robust and efficient storage architecture. *Science*, 355(6328):950–954, 2017.
- [11] D. Deamer, M. Akeson und D. Branton. Three decades of nanopore sequencing. *Nature Biotechnology*, 34:518–524, 2016.
- [12] L. Ceze, J. Nivala und K. Strauss. Molecular digital data storage using DNA. *Nature Reviews Genetics*, 20:456–466, 2019.